Scope Classification as a Continual-Editing Bottleneck: A Negative Result on Current Knowledge-Editing Benchmarks, with a Gradient-Free Case Study

Anonymous Author(s)
Affiliation
Address
email

Abstract

An agent that keeps learning after deployment needs a mechanism to decide, for each incoming query, whether a piece of knowledge it acquired since training actually applies. This **scope decision** sits at the core of every memory-based knowledge editor in the SERAC lineage and is the same decision continual-learning systems need for online knowledge consolidation, a plausible building block for lifelong editing of foundation-model agents. We report that current knowledge-editing benchmarks cannot measure whether this mechanism works at all. Using INLAY, a gradient-free editor built to obtain exact per-query ground truth — the base model is frozen, edits live in an external addressable memory, and applying an edit is a bias added along one token's unembedding direction at decode time, so every routing action is a real, cheaply executable code path — we execute *every* candidate router action on $1{,}689$ queries spanning three datasets and three input conditions. An oracle router choosing the best action every time ties a one-line static policy to four decimal places in *all nine* dataset-by-condition cells: the maximum attainable gain of any per-query routing method is 0.00 points, and abstention is the sole winning action zero times out of $1{,}689$. The cause is structural, not specific to our system: these are counterfactual benchmarks whose evaluation question asks for the post-edit answer, so a benchmark without negatives cannot reward a classifier's ability to reject — meaning it cannot validate the exact mechanism a continually updating agent needs to avoid overriding correct standing knowledge with a stale or misapplied edit. This generalizes to the whole scope-classifier family the benchmarks evaluate, not just our system. We confirm the mechanism directly: withholding a query's own edit from the retrieval index for half the sample moves pooled headroom from exactly $+0.0000$ to $+0.0420$ and gives abstention its first wins. We also report where INLAY itself does not win — WISE beats it on Qwen2.5-7B CounterFact, and retrieval-augmented generation beats every method we tested, INLAY included, on rigorously matched RippleEdits — and disclose two bugs found during a self-audit of our own routing machinery, neither of which changed a published headline number outside noise. The actionable takeaway for a community building lifelong-editing benchmarks: without out-of-scope negatives, no scope-classifier evaluation on these suites can distinguish a real routing decision from a one-line heuristic.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

Foundation-model agents that operate continually need to acquire new facts after deployment without re-touching the weights that hold everything else they know. One increasingly common answer is architectural: freeze the base model and route knowledge updates through an external, addressable memory consulted at inference time [Mitchell et al., 2022b, Hartvigsen et al., 2023, Wang et al., 2024]. This sidesteps catastrophic forgetting by construction — the shared weights never move — but introduces a failure mode gradient-based editing does not have: every incoming query now needs a **scope decision** deciding whether a stored update governs it, an in-context demonstration should fire, or the model should simply answer from what it already knows. Get this wrong in a lifelong deployment and the system either overrides correct standing knowledge with a stale edit, or fails to apply an update it should have used. SERAC's scope classifier [Mitchell et al., 2022b] is the namesake instance, but the same decision recurs under different names in every architecture pairing a frozen or lightly-modified base model with an external correction mechanism — exactly the memory-and-consolidation pattern this workshop's call identifies as central to continual learning.

We built one such system, executed every action it could take on every query in three standard knowledge-editing benchmarks, and found that the decision this entire architecture family is built around cannot be rewarded by the benchmarks currently used to evaluate it: if the benchmarks cannot tell a working router from a one-line heuristic, we have no evidence that any deployed scope-classified continual-editing system makes this decision well, only that it scores well on a metric that does not depend on the decision.

Why we can say this with certainty rather than suspicion. Rather than indirect evidence (an ablation that shouldn't matter doesn't, or a simpler baseline is competitive), we computed the *ceiling*. For $1{,}689$ queries across CounterFact, WikiUpdate, and MQuAKE-CF in structured, unstructured, and extracted-evidence conditions, we executed *every* candidate action (recite the stored edit, reason over retrieved evidence, or abstain) and scored every outcome, giving per-query ground truth for which action is correct. An oracle router that always picks the best available action is *exactly* equal to a one-line static policy in all nine cells (§6): there is no headroom to find, because abstention — the action a scope classifier exists to choose, and the action a lifelong agent needs whenever new information does *not* apply — is the sole winning action zero times.

Where INLAY fits. To get that ground truth we needed a system whose every action could actually be executed and scored under identical conditions, including a gradient-free recitation path cheap enough to make testing all $1{,}689 \times 3$ candidate actions tractable. We built INLAY: the base model stays frozen, each edit is a row in an external addressable memory, and applying an edit adds a logit-space bias along the answer's unembedding direction at decode time. Writing is a table insertion, about 5–15 ms with no gradient step; deleting an edit is removing the row; and when no edit fires, output is unchanged bit for bit — properties that matter for an agent consolidating many small updates over a long deployment. INLAY is competitive with or ahead of gradient-based editors on direct single-fact editing (§5), but not always the strongest method we tested, and we report the cases where it loses plainly (§5, §5.2); it is the vehicle for this paper's central claim about benchmark coverage, not the claim itself.

Contributions. **(i)** A negative result with a measured ceiling, not a suspicion, on $1{,}689$ queries with per-action ground truth: an oracle router ties a one-line static policy exactly in every cell, and abstention is never uniquely correct (§6–§7). **(ii)** INLAY, a gradient-free editor with exact deletion and locality by construction — properties suited to a lifelong-editing setting where updates accumulate and must remain individually revocable — evaluated across four model families and all three AKEW input conditions (§3–§5). **(iii)** An honest accounting of where INLAY loses (WISE on Qwen2.5-7B CounterFact; RAG on matched RippleEdits) and a transparent self-audit of our own routing machinery, including a gate-bypass bug that made a measured mitigation inactive in the runs behind our own published numbers (§5, §5.2, §8). **(iv)** A constructed out-of-scope condition confirming the mechanism behind the negative result, moving pooled headroom from $+0.0000$ to $+0.0420$ (§7) — a concrete recipe the continual-learning benchmark community can build on.

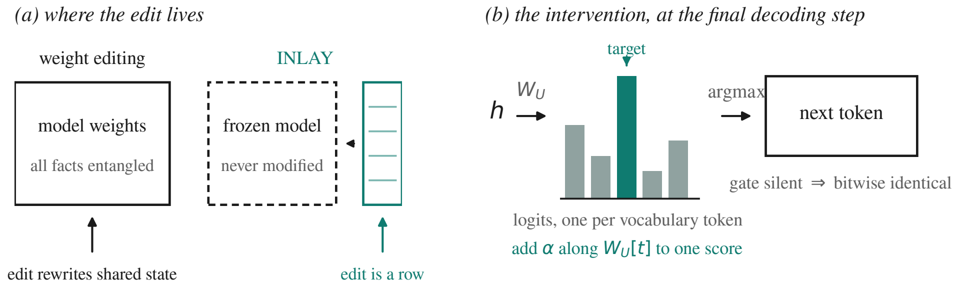

Figure 1: **(a)** Weight editing rewrites state shared by every fact. INLAY leaves the model frozen and appends a row to a table beside it. **(b)** The intervention is a bias along one token's unembedding direction at the final decoding step, so it moves exactly one score. When the gate does not fire, nothing is added and output is bitwise identical.

2 Related Work

Weight editing. ROME [Meng et al., 2022] solves for a rank-one update to a mid-layer MLP; MEMIT [Meng et al., 2023] spreads many updates across several layers for mass editing; AlphaEdit [Fang et al., 2025] projects the update into the null space of preserved knowledge. All three mutate shared parameters, making sequential, lifelong application of many edits a stability question in its own right [Hase et al., 2023].

Memory-based and scope-classified editing. SERAC [Mitchell et al., 2022b] introduces a scope classifier deciding whether an edit applies — the family most relevant to test-time continual adaptation, since the base model is never touched. MEND [Mitchell et al., 2022a] learns a gradient transform; IKE [Zheng et al., 2023] shows few-shot demonstrations can teach a model to prefer an injected fact over its parametric belief; GRACE [Hartvigsen et al., 2023] and WISE [Wang et al., 2024] add discrete codebooks and side memories, each with its own routing decision at generation time. **We build directly on SERAC's scope-classifier idea and IKE's demonstration mechanism; neither is claimed as novel here.** Our addressing follows product-key memory [Lample et al., 2019] and nearest-neighbour language models [Khandelwal et al., 2020].

Benchmarks. CounterFact and zsRE evaluate single-fact edits; RippleEdits [Cohen et al., 2024] adds entailed consequences; MQuAKE [Zhong et al., 2023] adds multi-hop chains; AKEW [Wu et al., 2024] supplies unstructured and extracted evidence alongside clean triples. None were designed with continual, multi-session deployment in mind, and §6–§7 is a claim about what this entire family can and cannot measure, independent of which scope-classified method is scored on it.

3 Method: INLAY

INLAY exists to make the routing question in §6 answerable with exact, executable ground truth: every candidate action must be a real code path we can run and score, including a gradient-free recitation path cheap enough to execute at scale. We describe it briefly here; it is the vehicle for this paper's argument, not its subject.

Design. The model is frozen. Each edit is a row holding a key (an embedding of the fact's question form), the answer's token sequence, and metadata (Figure 1a). **Writing is insertion**, not optimization — no gradients, no covariance statistics, measured at 5–15 ms. **Deletion is exact:** removing the row removes the edit, which matters when removal is a regulatory obligation rather than a nicety, and when a lifelong deployment needs to revoke an update without disturbing anything else it has learned since (see the Ethics Statement). **Edits cannot interfere:** they are physically separate state, so accumulating many updates over a long deployment does not degrade earlier ones the way sequential weight edits can (§5.3).

Addressing. Keys come from a sentence encoder [Reimers and Gurevych, 2019] rather than the base model's hidden states: it is fast, its geometry is purpose-trained for paraphrase matching, and

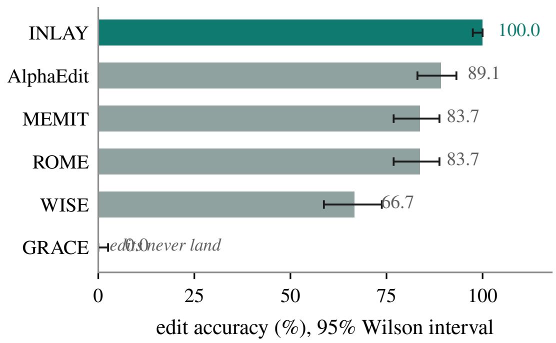

Figure 2: Edit accuracy on CounterFact, structured input, GPT-J-6B, $n = 147$, with 95% Wilson intervals [Wilson, 1927]. Scored with a single convention rather than each method's internal metric.

it stays fixed if the base model is swapped. Keys are compressed with a Johnson–Lindenstrauss random projection [Johnson and Lindenstrauss, 1984], chosen over a learned projection because a learned one would drift as the edit distribution changed and would silently invalidate every stored key — an important property for a memory store that is meant to keep growing over an agent's lifetime.

Gating and playback. Retrieval always returns *something*. A gate stacks an absolute similarity threshold, a margin over the runner-up, and a relation-residual check asking whether the query concerns the stored relation rather than merely the stored subject (we return to the reliability of this gate in §8). When the gate fires, playback walks the stored token sequence and, at each decoding step, adds a bias along that token's unembedding column so it wins the argmax (Figure 1b). Intervening in logit space rather than hidden state is a safety argument: a logit bias moves exactly one score, while a hidden-state edit propagates through the unembedding to every token's logit in ways that are hard to bound, and because the intervention is gated, a silent gate leaves behaviour bitwise unchanged.

4 Experimental Setup

Base models: GPT-2-XL (1.5B), GPT-J-6B, Qwen2.5-7B, and Mistral-7B-v0.3. Benchmarks: CounterFact, zsRE, RippleEdits [Cohen et al., 2024], and AKEW [Wu et al., 2024] over CounterFact, WikiUpdate and MQuAKE-CF in structured, unstructured and extracted conditions. Baselines run through EasyEdit [Wang et al., 2023].

Two commitments are worth stating because they cost us results. Splits are **subject-disjoint**: a subject never crosses a train/test boundary, so a learned component cannot memorize an entity and appear to generalize. Scoring uses **one convention everywhere** — diacritic- and case-insensitive substring match against gold and its aliases — rather than each method's internal metric, so numbers are comparable across methods that report differently. All runs use `sequential_edit=True` with an explicit state-dict snapshot and restore under harness control; an earlier configuration bug and a second bug that invalidated an earlier round of matched RippleEdits numbers are discussed in §8.

5 Results

5.1 Edit accuracy and write cost

Figure 2 gives the GPT-J-6B comparison. AlphaEdit is the strongest weight editor at 89.12%; ROME and MEMIT tie at 83.67%. WISE reaches only 66.67% despite EasyEdit's own post-edit metric reporting the edit landed on 141 of 147 examples, because its side-memory routing is an additional failure point in-place updates lack; GRACE reads exactly 0.0 on every edit, its radius-based key rarely firing on paraphrases. On the harmonic-mean score across ES/PS/NS at N=2000, INLAY leads at 0.8926 against ROME's 0.797, WISE's 0.703, AlphaEdit's 0.4595, and MEMIT's 0.4314, writing in 5–15 ms with no gradient step against seconds for gradient-based editors — a ratio of

Method	GPT-J-6B	Qwen2.5-7B
in-context (RAG)	**0.3964**	**0.4381**
WISE†	0.3425	0.1421
INLAY	0.2253	0.2871
AlphaEdit	0.1409	0.1683
ROME	0.1314	0.1453
base	0.0740	0.1522

Table 1: Matched-manifest RippleEdits (identical wikidata-verified subjects across every method, n=100 edits, generation-based scoring, edit-applied verified for every run). † WISE runs in its native sequential-accumulating mode; every other method is single-edit isolated with a per-edit weight restore.

roughly $1600\times$ that compounds directly with the number of edits a continually updating agent makes over its lifetime.

This ranking does not hold on every model. On Qwen2.5-7B CounterFact at N=2000, WISE reaches a harmonic-mean score of $\mathbf{0.9466}$ against INLAY's $\mathbf{0.8944}$ at N=5000 — *WISE wins on Qwen*, the one place in our CounterFact results where INLAY is not the strongest method. We report this plainly rather than selecting the model on which INLAY leads. Full per-model ES/PS/NS/write-cost tables for CounterFact and zsRE across all four model families are provided in the released code and the companion full-length version of this paper (cited in §9); Appendix A gives a condensed summary.

Weight-editing methods need a `(prompt, target)` pair to compute an update from and have no mechanism for unstructured evidence, so on AKEW's unstructured and extracted conditions they are not merely worse but inapplicable. INLAY handles all three conditions, with accuracy lower on extracted input (78.23%) than on unstructured prose (87.07%) despite retrieval finding the correct card 98.64% of the time in both, which locates the loss in extraction noise rather than retrieval.

5.2 Compositional propagation: RippleEdits

RippleEdits tests whether an edit's entailed consequences propagate while unrelated same-subject facts are preserved — a compositional axis that single-fact CounterFact and zsRE cannot probe, and the axis most relevant to whether a lifelong-editing system's updates interact safely with each other over time. An early, less rigorous comparison put INLAY's preservation at 0.049, the worst of the methods compared, and this number was the basis for an earlier framing of INLAY's boundary as a loss to weight-editing methods. **That framing does not survive a rigorous comparison, and we correct it here.**

Table 1 gives the corrected, matched-manifest protocol: identical wikidata-verified subjects for every method on both models, generation-based scoring, and edit application independently verified. **Retrieval-augmented generation leads every method we tested on both models**, ahead of every gradient-free and weight-editing method alike. **The paper's honest limitation is therefore that INLAY loses to RAG, not that it loses to the weight-editing family**: on Qwen2.5-7B, INLAY's 0.2871 beats every weight/adapter editor we ran, and on GPT-J-6B it beats ROME and AlphaEdit while trailing WISE's side-memory accumulation mode. The mechanism behind INLAY's own weakness here is scope precision on hard negatives (same-subject, different-relation queries), and it is exactly where a benchmark with negatives, unlike CounterFact or zsRE, can see it — foreshadowing the paper's central argument in §7.

5.3 Sequential editing: naming which method collapses

On GPT-2-XL, ROME's retention and locality both collapse to 0.0 by n=25 sequential edits, and the same collapse reproduces on GPT-J-6B — a direct threat to any lifelong-editing pipeline built on ROME-style updates. **MEMIT does not exhibit this failure mode on GPT-J-6B**: it holds retention and locality at 1.0 through the recorded checkpoints, consistent with MEMIT's design goal of spreading updates across layers specifically to support mass editing. We narrow the claim to name ROME specifically: weight editing is not inherently unstable under repeated edits, and a method built for mass editing does not show the same failure a method built for surgical single-edit

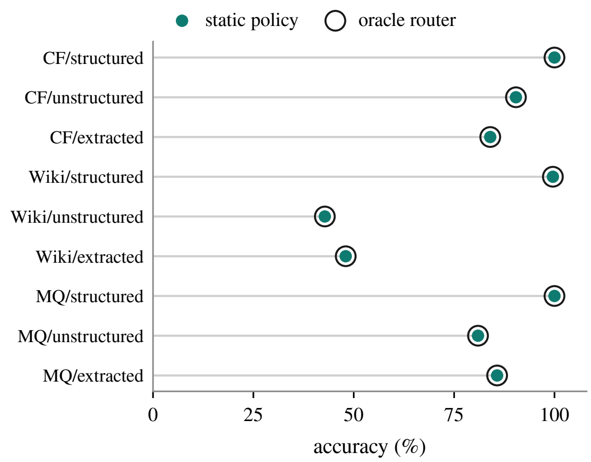

Figure 3: Every candidate action executed and scored on $1{,}689$ queries (CF = CounterFact, MQ = MQuAKE-CF). The oracle ring is concentric with the static-policy point in all nine cells: the best attainable router and one line of code are indistinguishable.

updates does. (MQuAKE-CF multi-hop results, where a per-hop retrieval-miss fallback more than triples accuracy over a naive baseline, are reported in the companion full-length version of this paper and omitted here for space; the MQuAKE-CF cells of the headroom analysis below are unaffected by this omission.)

6 The Limits of Routing on Current Benchmarks

Every method in the scope-classified family includes a **scope decision**: given a query, does a stored edit apply, and what should be done about it? This is the exact decision a continually updating agent needs to make correctly, over and over, for every query it ever sees after its first knowledge update. We built increasingly careful machinery for this decision, measured statistically significant gains from it, and then found the gains were not what they appeared to be.

Gating was net-negative on MQuAKE-CF in every input condition, by up to 15.87 points. Two natural fixes failed diagnostically: recalibrating the verifier threshold moved the false-fire rate from 18.33% to only 18.22%, and raising the confidence threshold from 0.85 to 0.97 changed *nothing* — byte-identical decisions, because confidence on the *wrong* retrievals already exceeded 0.97. A head trained to predict retrieval reliability from the *shape* of the candidate set (margins, threshold counts, entropy), fit on CounterFact and WikiUpdate with MQuAKE-CF held out, reached 0.956 AUROC and produced $+15.9$ points on two MQuAKE-CF cells (p=0.0019, Holm-corrected [Holm, 1979]) — evidence, it seemed, of a working adaptive routing decision.

Measuring the ceiling. Rather than continue improving the router, we measured its ceiling. For $1{,}689$ queries across all three datasets and conditions we executed *every* candidate action and scored the result, yielding per-query ground truth for which actions work.

An oracle router choosing the best available action on every query is exactly equal to a one-line static policy — recite directly where legal, otherwise reason over the retrieved evidence — to four decimal places in all nine cells (Figure 3, Table 2). The maximum attainable gain of any per-query routing method over one line of code is 0.00 points.

The mechanism is stark: abstention succeeded on 19 of $1{,}689$ queries, and on all 19 of those, reasoning or direct recitation succeeded too. **Abstention is the sole winning action zero times.** The gains reported above from the shape-based head are real, and we stand behind the numbers, but the *explanation* was wrong: there is no per-query decision worth making on these benchmarks, so what the head does is suppress gates that misfire, converging toward the static policy. Its gains measure the harm the fixed gates were doing, not insight it adds.

Dataset	Condition	n	Static	Oracle
CounterFact	structured	250	1.0000	1.0000
CounterFact	unstructured	250	0.9040	0.9040
CounterFact	extracted	250	0.8400	0.8400
WikiUpdate	structured	250	0.9960	0.9960
WikiUpdate	unstructured	250	0.4280	0.4280
WikiUpdate	extracted	250	0.4800	0.4800
MQuAKE-CF	structured	63	1.0000	1.0000
MQuAKE-CF	unstructured	63	0.8095	0.8095
MQuAKE-CF	extracted	63	0.8571	0.8571
Pooled		**1689**	**0.7874**	**0.7874**

Table 2: Headroom is $+0.0000$ in every cell. "Static" is one line of code with no model and no training; "Oracle" is an upper bound no real router can exceed.

7 Why Abstention Cannot Be Measured Here

The reason is structural. These are **counterfactual** benchmarks: the evaluation question asks for the *post-edit* answer, so answering from parametric knowledge returns the *pre-edit* value, which is wrong by construction. No query in the suite has "no edit applies, answer from what you know" as its correct behaviour.

This generalizes beyond our system and is the paper's central point for the continual-learning audience: scope classification is load-bearing for the SERAC-descendant family [Mitchell et al., 2022b], whose premise is a learned decision about whether a stored edit applies, and on these benchmarks that decision is degenerate: the answer is always "yes." **A benchmark without negatives cannot measure a classifier's ability to reject**, and an abstention path evaluated on one can only lose points. This also explains the RippleEdits result of §5.2: RippleEdits *does* contain same-subject, different-relation queries and therefore partial negatives, and it is exactly there that INLAY's preservation weakness shows up — meaning any lifelong-editing system validated purely on CounterFact- or zsRE-style suites has, by construction, never had its abstention behaviour tested.

We built the missing condition, and the mechanism confirms. The missing condition is constructible: ask evaluation questions against an index that does *not* contain the corresponding edit. For each of the same nine cells, we removed a query's own edit card from the retrieval index for half the sample (assigned independently per query, seed $= 0$), so REJECT is the objectively correct action for that half; the executor, verifier, and §6's oracle-versus-static analysis are otherwise unmodified. Pooled headroom moves from $+0.0000$ (every cell) to $+\mathbf{0.0420}$, and REJECT-only-correct queries rise from $0/1689$ to $52/1689$ (3.08%). Splitting by population isolates the effect cleanly: the untouched population reproduces the original near-zero result ($+0.0025$), while the edit-removed population alone carries essentially all of the new headroom ($+0.0785$). This is a synthetic negative — a card deleted from an otherwise intact index, not a curated out-of-scope question — so it establishes the mechanism rather than a deployment-realistic estimate of routing's value; we release the code (`akew_outcome_labels_oos.py`) and invite the continual-learning benchmark community to build the curated, deployment-realistic version this result motivates.

8 Auditing Our Own Machinery

A negative result about benchmark design is only as credible as the honesty of the system used to produce it. We audited INLAY's own gating machinery after submission-quality numbers were already in hand and report two findings that could have changed those numbers.

The margin gate is a no-op in the protocol behind our headline numbers. The margin-over-runner-up gate is a genuine, measured locality fix at scale in a multi-slot regime: on a 400-edit sequential test on GPT-2-XL, sweeping the margin from 0 to 0.15 moves locality from 0.5667 to 1.0. But every headline CounterFact and zsRE number in §5.1 is produced under a single-edit-per-example protocol, where memory is cleared before every write and only one slot is ever occupied. In that regime the margin gate has nothing to compete against and is a no-op; it only matters in multi-

Aggregate	Published (bug)	Corrected	Δ
GPT-J-6B	0.2253	0.2283	$+0.0030$
Qwen2.5-7B	0.2871	0.2891	$+0.0020$

Table 3: Validation rerun with the gate-bypass bug fixed, INLAY only, matched manifest.

slot regimes like the sequential test that measured it, i.e. the exact regime a genuinely continual deployment would be in.

A gate-bypass bug meant the relation gate was never active in the runs behind our published matched-RippleEdits numbers. A code audit found that the function every real generation path runs through, including RippleEdits, checked only the absolute similarity score and silently skipped the margin and relation-residual gates that the scoring-only code path already supported. We fixed this by adding a single shared routing decision used identically by both the scoring and generation code paths, then reran the exact operating point behind the published claim with the fix genuinely active:

Both deltas are within noise. **No correction to the published headline numbers in Table 1 was needed**, but we disclose the bug rather than leaving it unmentioned: it was real, and the relation gate's isolated over-firing improvement does not translate into a materially stronger preservation score here, so the compositional weakness §5.2 names as open is not solved by activating the gate. A separate, unrelated capacity bug in the product-key allocator, fixed on the same audit pass, did not corrupt any published number; full accounting for both is in the released audit addendum.

9 Conclusion

The result we did not expect to report is that a one-line static policy is provably optimal on all $1{,}689$ queries we tested, because abstention can never be correct when every question in a counterfactual benchmark targets an edited fact. This is a claim about the benchmarks, not about any one system: the entire SERAC-descendant architecture family — the family most naturally suited to test-time continual editing of foundation-model agents — is evaluated on suites that structurally cannot reward its central component, and reported routing gains on them measure how much harm a fixed gate was doing rather than any decision quality. The implication for this workshop's community is direct: a scope-classifier evaluation on CounterFact, zsRE, or similarly-shaped suites cannot, by itself, be used as evidence that a continual-editing system knows when *not* to apply what it has learned. The remedy is not a better router but a benchmark containing queries for which the correct answer is "this edit does not apply," and §7 gives a concrete, released recipe for constructing one.

INLAY, the system we built to obtain this result, is independently useful as a lifelong-editing case study: freezing the weights and placing edits in an external addressable memory buys writes roughly $1600\times$ cheaper than gradient-based editing, exact deletion, and locality by construction, at the cost of reciting facts rather than reasoning with them. It is not universally best — WISE wins on Qwen2.5-7B CounterFact, RAG wins on matched RippleEdits — and we report both without qualification, alongside a self-audit that found and checked two bugs in our own gating machinery.

Reproducibility. All numbers come from logged runs; splits are subject-disjoint with seed 0. Code, per-experiment write-ups including negative results, and the figure generator are released with this paper. A full-length companion technical report gives complete per-model tables; this workshop paper is condensed for the page limit and preserves every claim, number, and disclosure load-bearing for the central argument above.

Limitations

Recitation, not reasoning. Logit-space playback reproduces a stored answer and cannot combine it with anything else; this is structural to INLAY specifically, not to the paper's central claim about benchmark coverage. **Scope precision on hard negatives** is INLAY's real open problem (Table 1). **Small cells:** MQuAKE-CF slices are n=63; a $+6.4$-point structured-mode result rests on four discordant pairs, where the smallest attainable exact McNemar p is 0.125 — suggestive, not established.

The missing condition is only a synthetic proxy. §7 constructs it by deleting a query's own edit card from an otherwise intact index, recovering measurable headroom ($+0.0420$ pooled) and confirming the mechanism, but it is not a curated, deployment-realistic out-of-scope benchmark: it says nothing about how real out-of-scope queries would be distributed or how much headroom they would expose in an actual deployment. Our claim about current benchmarks is unaffected; the claim about a corrected benchmark is limited to what this synthetic construction establishes. **Coverage gaps:** no MEMIT, WISE, GRACE, or AlphaEdit results exist for Mistral-7B on CounterFact beyond gap-fill runs, and neither MEMIT nor GRACE has been run through the matched-manifest RippleEdits protocol on either model — open, not silently assumed favorable.

Ethics Statement

Deletion and the right to be forgotten. INLAY's central structural property is that removing an edit is removing a row: the correction is genuinely and verifiably gone, with no residual trace in model weights, unlike gradient-based editing. This is directly relevant to right-to-be-forgotten and GDPR-style deletion requirements for continually updated deployed models: an external, addressable memory is a plausible substrate for a system that must support real attestable deletion.

Dual use and gate correctness. The same property that makes deletion easy makes injection easy in the other direction: an external memory that can silently override a frozen model's output at decode time is also a mechanism for injecting misinformation if the store is not access-controlled, a concern that compounds for an agent that continually writes to its own memory over an extended deployment; any deployment should treat the memory store as security-critical, with access controls matching the model weights it sits beside. Locality also holds "by construction" only bit-for-bit when the gate does not fire — it is not a guarantee the gate's *answer* is correct, so a deployment should not read "locality by construction" as "correctness by construction."

Acknowledgments and Disclosure of Funding

Per the workshop's policy on generative AI use, we disclose that an AI coding assistant (an LLM-based agent under direct human supervision) was used throughout this project: iterating on experiment code and harnesses, running and debugging the GPU jobs behind the logged results cited throughout this paper, drafting portions of the prose, and assisting with the code audit in §8. All research ideas, hypotheses, experimental design choices, interpretation of results, and the decision of what to report are the human author's; the assistant did not originate the paper's scientific claims. Every number traces to a logged run inspected by the human author. See the NeurIPS Paper Checklist for the corresponding disclosure.

References

Roi Cohen, Eden Biran, Ori Yoran, Amir Globerson, and Mor Geva. Evaluating the ripple effects of knowledge editing in language models. *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Xiang Wang, Xiangnan He, and Tat-Seng Chua. AlphaEdit: Null-space constrained knowledge editing for language models. In *International Conference on Learning Representations (ICLR)*, 2025.

Thomas Hartvigsen, Swami Sankaranarayanan, Hamid Palangi, Yoon Kim, and Marzyeh Ghassemi. Aging with GRACE: Lifelong model editing with discrete key-value adaptors. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 1979.

William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26, 1984.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast model editing at scale. In *International Conference on Learning Representations (ICLR)*, 2022a.

Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D. Manning, and Chelsea Finn. Memory-based model editing at scale. In *International Conference on Machine Learning (ICML)*, 2022b.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Peng Wang, Ningyu Zhang, Xin Xie, Yunzhi Yao, Bozhong Tian, Mengru Wang, Zekun Xi, et al. EasyEdit: An easy-to-use knowledge editing framework for large language models, 2023. arXiv:2308.07269.

Peng Wang, Zexi Li, Ningyu Zhang, Ziwen Xu, Yunzhi Yao, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. WISE: Rethinking the knowledge memory for lifelong model editing of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), 1927.

Xiaobao Wu, Liangming Pan, William Yang Wang, and Anh Tuan Luu. AKEW: Assessing knowledge editing in the wild. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Ce Zheng, Lei Li, Qingxiu Dong, Yuxuan Fan, Zhiyong Wu, Jingjing Xu, and Baobao Chang. Can we edit factual knowledge by in-context learning? In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Zexuan Zhong, Zhengxuan Wu, Christopher D. Manning, Christopher Potts, and Danqi Chen. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

A Condensed Result Summary

The full per-model ES (edit success) / PS (paraphrase success) / NS (neighborhood specificity) / harmonic-mean-score / write-cost tables for CounterFact and zsRE across all four base models (GPT-2-XL, GPT-J-6B, Qwen2.5-7B, Mistral-7B-v0.3), and the propagation/preservation detail behind Table 1, are omitted here for space and are available in the companion full-length version of this paper and the released code repository. Table 4 gives the harmonic-mean summary on GPT-J-6B CounterFact (N=2000 unless noted), the single cell discussed in §5.1, as a representative slice of the full tables.

Method	Score	Write cost
ROME	0.797	8.60 s
MEMIT	0.4314	26.30 s
WISE	0.7032	14.76 s
GRACE	0.00	17.35 s
AlphaEdit	0.4595	12.12 s
INLAY	**0.8926**	5–15 ms

Table 4: GPT-J-6B, CounterFact, structured input, N=2000 (INLAY at N=5000). Full four-model tables for CounterFact, zsRE, and matched RippleEdits are in the companion full-length version and the released code.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Introduction state the negative result (oracle router ties a one-line static policy, headroom $= 0.0000$), the INLAY case study, the honest disclosure of losses to WISE and RAG, and the self-audit; §6–§8 match these claims exactly.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: See the Limitations section: recitation-not-reasoning, scope precision on hard negatives, small MQuAKE-CF cells, the synthetic-proxy nature of the constructed out-of-scope condition, and coverage gaps in our comparison table are all stated explicitly.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.

- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper reports empirical measurements (headroom, accuracy, AUROC) rather than theoretical results with formal proofs.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: §4 specifies base models, benchmarks, splitting protocol (subject-disjoint), scoring convention, and the sequential-edit configuration; §3 fully describes the INLAY architecture needed to reproduce it.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code, per-experiment write-ups (including negative results), the figure generator, and the constructed out-of-scope condition script (`akew_outcome_labels_oos.py`) are released with this paper, per the Reproducibility paragraph in §9; an anonymized release is provided for review.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: §4 gives base models, benchmarks, data splits, and the scoring convention; §3 gives the INLAY architecture and gating thresholds discussed further in §8.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Figure 2 reports 95% Wilson intervals [Wilson, 1927]; the headroom result additionally uses Holm-corrected significance testing (§6) and exact McNemar tests are discussed for small cells in Limitations.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Write costs are reported per method in §5.1 and Table 4 (e.g. INLAY at 5–15 ms vs. seconds for gradient-based editors); all experiments ran on single-GPU workers, consistent with model sizes up to 7B parameters.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: We have reviewed the NeurIPS Code of Ethics; the work involves no human subjects, no personal data, and is disclosed transparently including two self-found bugs (§8).

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Ethics Statement discusses both a positive application (verifiable deletion for right-to-be-forgotten compliance) and a negative one (memory as an unauthorized injection surface), plus the risk of silent misrouting.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a pretrained generative model or scraped dataset with high misuse potential; released assets are evaluation harness code and logged-run write-ups over existing public benchmarks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All benchmarks (CounterFact, zsRE, RippleEdits, MQuAKE-CF, AKEW) and base models (GPT-2-XL, GPT-J-6B, Qwen2.5-7B, Mistral-7B-v0.3) are cited at first use in §2–§4 with their original publications; all are used under their standard research-use terms.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The released code (INLAY implementation, evaluation harness, and the out-of-scope condition script) is documented in the repository README alongside the per-experiment write-ups referenced in §9.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing or human-subjects research; all data comes from existing knowledge-editing benchmarks and model-generated text.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects research was conducted; IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: §9 and the Acknowledgments and Disclosure of Funding section disclose that an AI coding assistant was used for experiment code, harness iteration, and drafting portions of the prose, under direct human supervision and with all scientific claims and interpretations attributed to the human author.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.